

RISC-V 技术文档 Demo

葛良晨

2026 年 5 月 15 日

目录

0.1 修订历史	ii
第一章 Roadmap of Linux Kernel	1
1.1 系统使用与管理	1
1.1.1 进程与服务管理	1
第二章 Nix 包管理	5
第三章 IC 设计相关知识积累	6
3.1 一些易用设置	6
第四章 Hermes Agent 相关知识	7
4.1 rsync 文件同步工具	7
4.1.1 常用选项	7
4.1.2 Hermes 数据同步中的应用	7
4.1.3 与 cp/scp 的对比	7
4.2 LLM 对话原理：上下文与“模拟对话”	7
4.2.1 核心认知	7
4.2.2 每次请求都发送完整历史	8
4.2.3 对话是模拟出来的	8
4.2.4 KV Cache 与 Prompt Caching	8
4.2.5 启示	9
4.3 Hermes Feishu 网关连接与用户白名单 Debug 记录	9
4.3.1 问题描述	9
4.3.2 环境	9
4.3.3 日志现象	9
4.3.4 原因分析	9
4.3.5 解决步骤	10
4.3.6 验证	11
4.3.7 关键文件路径	11
4.3.8 飞书相关环境变量	11
4.4 DeepSeek API 消费监控	11
附录 A 一些碎碎念	13

0.1 修订历史

1.0 葛良晨 (2025-05-4)

Added

- L^AT_EX 模版搭建完成

第一章 Roadmap of Linux Kernel

Linux 学习的 Roadmap，主要包括三大主题（系统的使用与管理，核心子系统原理，内核深度剖析），以及一个额外的高级主题。

- **系统的使用与管理**：这一部分不是简单介绍基础操作。而是比价深入的理解各种基础概念。
 - **进程与服务管理**：使用 ps/top/htop 等工具，相关的系统调用 fork/exec/wait 等；服务管理 systemd/units；Linux 的启动流程演化路径。
 - **用户/权限/安全模型**：UID/GID/权限位/ACL；sudo 机制，PAM（安全）capabilities（替代 root）；Linux 安全模块（LSM）和 SELinux/AppArmor 等安全框架；Linux 的安全加固和最佳实践。
 - **文件系统与存储管理**：VFS 抽象；常见 FS ext4/xfs/btrfs/procfs/sysfs；mount / namespace。存储上，block device；LVM；RAID；buffer vs cache；direct IO/ buffered IO；IO 调度器（CFQ/Deadline/NOOP）；
 - **网络管理**：socket 编程；TCP/IP；ss/netstat；iproute2 工具；iptables/nftables；network namespaces；
 - **系统监控与性能分析**：iostat/vmstat/pidstat；perf 工具；strace/ltrace/bpfttrace；systemtap/eBPF；日志管理（syslog/journald/dmesg）；
- **核心子系统原理**：这一部分深入探讨 Linux 内核的核心子系统，包括进程管理、内存管理、文件系统、网络协议栈等。通过理解这些子系统的原理，读者可以更好地理解 Linux 内核的工作机制。
- **内核深度剖析**：这一部分将深入分析 Linux 内核的实现细节，包括内核模块开发、调试技巧、性能优化等内容。通过实际的代码示例和案例分析，读者可以掌握内核开发和调试的技能。
- **高级主题**：这一部分将涵盖一些高级主题，如实时内核、嵌入式 Linux、安全性增强等。这些内容将帮助读者了解 Linux 在特定领域的应用和发展趋势。

1.1 系统使用与管理

1.1.1 进程与服务管理

启动流程

启动流程 Linux 的启动流程可以分为以下几个阶段：

- **BIOS/UEFI 阶段**：计算机开机后，BIOS 或 UEFI 固件会进行硬件初始化，并寻找可引导设备。找到后，加载引导加载程序（如 GRUB）到内存并执行
- **引导加载程序阶段**：引导加载程序负责加载 Linux 内核和初始 RAM 盘（initrd/initramfs）到内存，并将控制权转交给内核。

- **内核初始化阶段：**内核开始执行，进行硬件检测和初始化，挂载根文件系统，并启动第一个用户空间进程（通常是 `init` 或 `systemd`）。内核初始化完成后，内核会调用 `init` 进程，通常是通过 `execve("/sbin/init", ...)`，这样第一个用户态进程（PID 1）就开始运行。
- **用户空间初始化阶段：**`init` 或 `systemd` 负责启动系统服务和用户进程，完成系统的初始化。

展开说说内核的启动过程 Bootloader（比如 GRUB）完成的工作有：

- 加载内核映像（`vmlinuz`）和初始 RAM 盘（`initrd/initramfs`）到内存。
- 设置内核启动参数 `cmdline`（如 `root=`、`console=` 等）。
- 将控制权转交给内核，通常是通过跳转到内核入口点（如 `0x100000`）来实现。

下面就进入到内核启动流程：

1. **体系结构相关的初始化：**内核会进行 CPU 特定的初始化，建立最初的栈，如设置 GDT/IDT、启用分页、初始化中断控制器、设置 CPU 模式等。
2. **`start_kernel` 函数：**这是内核的主入口点，负责调用各种初始化函数来设置内核环境。
3. **内核子系统初始化：**内核会依次初始化内存管理、调度器、中断子系统、定时器、设备模型、驱动框架和文件系统等核心模块，并完成根文件系统的挂载准备。

为什么要挂载 `initramfs`？

如何创建 `kernel_thread` 的？内核 PID 1 是如何变成用户态的？

`initramfs->rootfs` 切换是如何进行的？

内核的概念 内核用于整个操作系统核心，用于管理资源（CPU，内存，网络，存储，系统调用等）。内核是资源的拥有者和仲裁者。指责包括：

- **进程调度：**分配 CPU 时间片，管理进程状态（就绪、运行、阻塞等）。
- **内存管理：**分配和回收内存，管理虚拟内存和物理内存。
- **文件系统：**提供统一的接口访问不同类型的文件系统。
- **网络协议栈：**处理网络通信，支持各种协议（TCP/IP 等）。
- **设备驱动：**管理硬件设备的访问和控制。
- **系统调用：**提供用户空间与内核空间的接口，允许用户程序请求内核服务。

内核的主要工作方式：事件驱动 + 被动执行 + 少量主动线程

内核的代码运行在 3 种 context 中：

- **用户空间进程：**当用户空间的程序执行时，它会通过系统调用进入内核态，内核会执行相应的内核代码来处理请求。
- **中断处理程序：**当硬件设备发出中断信号时，内核会暂停当前正在执行的代码，转而执行相应的中断处理程序来响应硬件事件。

- **内核线程**：内核可以创建自己的线程（kernel thread），这些线程在内核态运行，执行内核任务，如处理网络请求、管理内存等。

Listing 1.1: Linux 内核的核心子系统

Linux Kernel

- 进程调度（Scheduler）
- 内存管理（Memory Management）
- 虚拟文件系统（VFS）
- 设备驱动（Drivers）
- 网络子系统（Networking）
- 系统调用接口（Syscall）
- 安全机制（LSM / 权限）

内核中的执行体 假设一个单核 CPU，它每时每刻都在执行指令，那么这个指令来自于哪里呢？有这样几种可能：

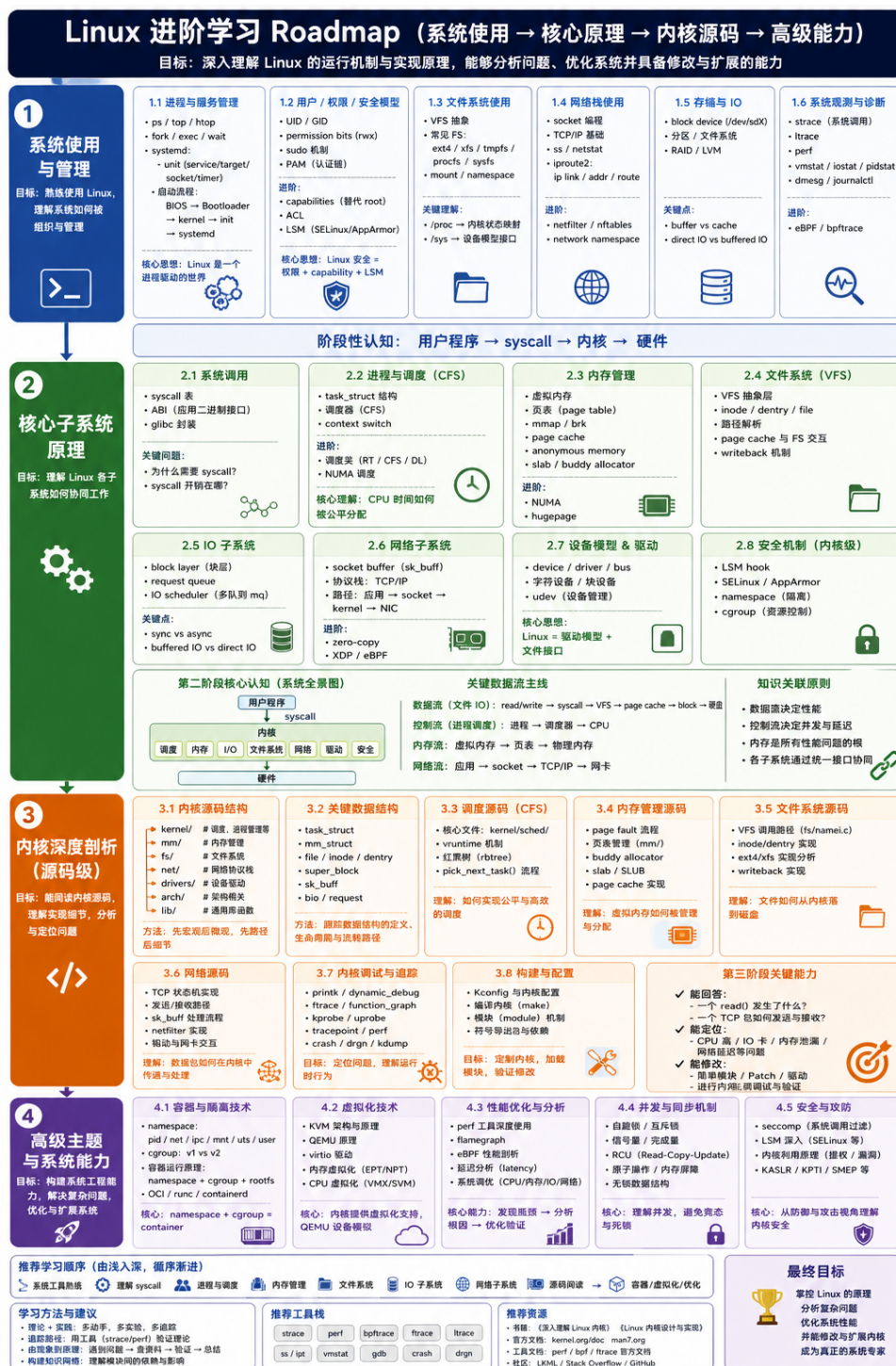
- **用户空间进程**：当用户空间的程序执行时，它会通过系统调用进入内核态，内核会执行相应的内核代码来处理请求。
- **内核 syscall**：当用户空间进程调用系统调用时，内核会执行相应的系统调用处理函数来完成用户请求。
- **内核线程**：内核可以创建自己的线程（kernel thread），这些线程在内核态运行，执行内核任务，如处理网络请求、管理内存等。
- **中断处理程序**：当硬件设备发出中断信号时，内核会暂停当前正在执行的代码，转而执行相应的中断处理程序来响应硬件事件。

现代 Linux 发行版已经从原本的 `init+etc/rc*.d` 脚本的启动方式 SysV，转变到了使用 `systemd`。启动模型也从原本串行执行脚本，变成依赖驱动的并行服务调度。

PID 1 的 `init` 进程是系统启动后第一个运行的用户空间进程，负责启动和管理其他系统服务和用户进程。它是系统的根进程，所有其他进程都是它的子孙进程。它具有如下的特点：

- **祖先进程**：所有其他进程都是 `init` 的子孙进程。当一个进程的父进程终止时，`init` 会接管这些孤儿进程。否则这些孤儿进程将成为僵尸进程，占用系统资源。
- **信号语义特殊**：`init` 进程对某些信号有特殊的处理方式。例如，`SIGTERM` 信号会被 `init` 忽略，而不是终止进程。这是为了确保系统的稳定性和可靠性。
- **系统初始化和服务管理**：`init` 负责启动和管理系统服务。比如，挂载为难系统，启动服务，设置环境。它会根据配置文件（如 `/etc/inittab` 或 `systemd` 的 `unit` 文件）来启动必要的服务，并确保它们在系统运行期间保持运行。

将内核和 PID 1 进行对比，重点是内核管理资源，PID 1 管理进程和服务。内核类似于政府 + 法律，PID 1 类似于总理，负责执行和管理系统服务，组织社会的运行。首相也不能违法，他在用户层，必须通过 `syscall` 来请求内核服务。



第二章 Nix 包管理

这个章节介绍了 Nix 包管理器的基本使用方法和一些常见的命令，同样也涉及了 NixOS 的相关内容。包含在 macOS 上使用 Nix 包管理器的说明。

第三章 IC 设计相关知识积累

Testbench 阶段标注 为了在 Verilog 中的仿真波形中看到目前测试的执行阶段，我们可以在 Testbench 中添加一些标志位或者输出语句来标注当前的阶段。例如：

```
typedef enum int {
    ST_INIT    = 0,
    ST_REG_RW  = 1,
    ST_SPLIT   = 2,
    ST_WSTRB   = 3,
    ST_STREAM  = 4,
    ST_DONE    = 5
} test_stage_e;
test_stage_e test_stage_id;

task automatic set_stage(input test_stage_e id, input string name);
begin
    test_stage_id = id;
    test_stage = name;
end
endtask
```

3.1 一些易用设置

Verdi 缩放波形居中显示 在使用 Verdi 进行仿真波形查看时，可以通过以下设置来缩放波形并居中显示：在 nWave 窗口中，选择 Waveform -> Keep Cursor at Center，这样在缩放波形时，光标会保持在中心位置，方便观察波形的细节。

第四章 Hermes Agent 相关知识

4.1 rsync 文件同步工具

rsync (remote sync) 是 Linux/macOS 上的文件同步工具，核心特点是**增量复制**——只传输有差异的部分，而非每次都全量拷贝。

4.1.1 常用选项

选项	含义
-a	archive 模式：递归 + 保留权限/时间/属主/符号链接等
--delete	删除目标端存在但源端已不存在的文件
-v	verbose，显示详细信息
-z	传输时压缩
--dry-run	模拟运行，不实际复制

4.1.2 Hermes 数据同步中的应用

在 Hermes 数据同步脚本中，其用法如下：

```
rsync -a --delete "$HERMES_DIR/skills/" "$REPO_DIR/skills/"
```

这表示：递归同步 `~/.hermes/skills/` 到仓库的 `skills/` 目录，保持文件属性不变，并删除仓库里本地已被删除的多余文件。

4.1.3 与 cp/scp 的对比

- **cp**：每次都全量复制，不论文件是否变更
- **scp**：跨网络全量复制，不支持断点续传
- **rsync**：仅传输差异部分，支持增量、断点续传

简言之，rsync 是智能版的 cp/备份工具。

4.2 LLM 对话原理：上下文与“模拟对话”

4.2.1 核心认知

LLM（大语言模型）与生俱来的能力只有一项：给定一段文本，预测下一个最合理的 token。所谓的“对话”、“记忆”、“上下文理解”全都是 prompt engineering 包装出来的产物。

4.2.2 每次请求都发送完整历史

每次用户发一条消息，Hermes（或其他 LLM 客户端）做的事：

用户输入："给我配飞书"

| Hermes 组装 prompt

[System Prompt (角色设定)]

[环境信息 (OS, CWD, ...)]

[持久化记忆 (user preferences)]

[历史对话 (压缩后)]

- 用户：配置飞书
- 助手：查日志 -> 发现 lark-oapi 缺失 -> 安装
- 用户：删掉错误 user
- 助手：改 .env -> 重启

[工具调用结果]

[本轮新消息："给我配飞书"]

| 发送给 LLM API (几十万 token)

LLM 做文本接龙 -> 返回回复

用户只发了几个字节。客户端（Hermes）拼了几十万 token 发给 API。

4.2.3 对话是模拟出来的

你以为的	实际发生的
模型在“和你对话”	模型在做文本接龙
模型“记得”你刚才说了什么	那段文本就在输入里
对话是原生模式	对话是 prompt 格式的产物
模型有“意图”	模型只是在模仿训练数据里的对话模式

关键创新是 **ChatML 格式**（由 OpenAI 提出）：

user：法国的首都都是哪里？

assistant：法国的首都都是巴黎。

user：那日本呢？

assistant：[LLM 在这里做文本接龙]

模型在训练时见过大量这种格式，学会了在这种格式下应该如何“扮演”。

4.2.4 KV Cache 与 Prompt Caching

每次发消息确实需要重新处理整个 prompt，但有两个优化：

KV Cache（推理引擎层面）

- **Prefill 阶段**：把完整 prompt 一次性输入，计算所有 token 的 KV cache（主要计算瓶颈）
- **Decode 阶段**：每吐一个 token，只需要增量计算最后一个 token 的 KV cache，复用前面的

Prompt Caching (API 层面)

- 无缓存：每次重新算全部历史的 KV，全部按 input token 计费
- 有缓存：重复部分按约 10% 缓存费率，新内容按全价

DeepSeek、Claude、Gemini、OpenAI 等主流模型均支持。

4.2.5 启示

1. 对话不是 LLM 的原生能力，而是通过数据格式和 prompt 设计模拟出来的
2. 每次请求都发送完整历史（包括所有历史对话 + system prompt + 环境信息）
3. 所谓“记忆力好”只是因为每次的输入里都包含了全部历史文本
4. 用户端的流量很小（几字节），但到 API 的流量很大（几十万 token）

4.3 Hermes Feishu 网关连接与用户白名单 Debug 记录

4.3.1 问题描述

Hermes Agent 飞书（Feishu/Lark）网关启动后无法回复用户消息。

- 在飞书上发消息给机器人，没有收到任何回复
- Gateway 日志中显示飞书适配器未加载

4.3.2 环境

- macOS 15.6
- Hermes Agent v0.13.0 (Nix 安装)
- Python 3.12.13 (Nix 环境)
- 飞书连接模式：WebSocket
- Feishu App ID: cli_aa8f26395378dbdb

4.3.3 日志现象

```
WARNING gateway.run: Feishu: lark-oapi not installed or FEISHU_APP_ID/SECRET not set
WARNING gateway.run: No adapter available for feishu
```

4.3.4 原因分析

原因 1: lark-oapi 库未安装

Hermes Gateway 使用 Nix 环境中的 Python 3.12(路径: `/nix/store/...-hermes-agent-env/bin/python`), 而 `lark-oapi` 库不存在于该环境中。飞书适配器在 `import lark_oapi` 失败时将 `FEISHU_AVAILABLE` 置为 `False`, 导致适配器无法启动。

原因 2: launchd 缺少 PYTHONPATH

即使安装了库, Gateway 的 launchd plist (`~/Library/LaunchAgents/ai.hermes.gateway.plist`) 中没有设置 `PYTHONPATH` 环境变量, Python 找不到额外安装的包。

原因 3: 用户白名单限制

`.env` 中 `FEISHU_ALLOW_ALL_USERS=false`, 初始白名单 `FEISHU_ALLOWED_USERS=b5g9a7g9` 中没有包含实际发消息的用户。日志中表现为:

```
WARNING gateway.run: Unauthorized user: ou_xxx (None) on feishu
```

4.3.5 解决步骤

1. 安装 lark-oapi 到 Nix Python 可访问的路径

由于 Nix 环境是只读的, `lark-oapi` 无法通过 `pip install` 直接安装到 Nix store 中的 `site-packages`。采用以下方案:

```
# 使用 macOS 系统 Python 的 pip 安装到 ~/.hermes/site-packages/
# 指定 Python 3.12 编译的 wheel 以保证兼容性
python3 -m pip install \
  --target ~/.hermes/site-packages \
  --only-binary=:all: \
  --python-version 3.12 \
  --platform macosx_11_0_arm64 \
  --abi cp312 \
  lark-oapi
```

2. 在 launchd plist 中添加 PYTHONPATH

编辑 `~/Library/LaunchAgents/ai.hermes.gateway.plist`, 在 `<key>EnvironmentVariables</key>` 的 `<dict>` 中添加:

```
<key>PYTHONPATH</key>
<string>/Users/katsu/.hermes/site-packages</string>
```

然后重新加载 plist:

```
launchctl unload ~/Library/LaunchAgents/ai.hermes.gateway.plist
launchctl load ~/Library/LaunchAgents/ai.hermes.gateway.plist
```

3. 更新用户白名单

在 `.env` 中找到 `FEISHU_ALLOWED_USERS`, 将新用户的 `open_id` 追加到逗号分隔的列表中:

```
FEISHU_ALLOWED_USERS=ou_8f29366d98158fbc2b52c7772dba32de
```

然后重启网关使其生效:

```
launchctl stop ai.hermes.gateway
sleep 2
launchctl start ai.hermes.gateway
```

4.3.6 验证

重启后日志显示飞书正常连接：

```
INFO gateway.run:    feishu connected
[Lark] [INFO] connected to wss://msg-frontier.feishu.cn/ws/v2?...
```

4.3.7 关键文件路径

文件	用途
~/hermes/.env	Feishu 配置 (APP_ID, APP_SECRET, 白名单等)
~/hermes/site-packages/	手动安装的 lark-oapi 库 (Nix Python 通过 PYTHONPATH)
~/Library/LaunchAgents/ai.hermes.gateway.plist	Gateway 的 launchd 服务配置
~/hermes/logs/gateway.log	网关运行日志

4.3.8 飞书相关环境变量

变量	说明
FEISHU_APP_ID	飞书应用 ID (cli_xxx)
FEISHU_APP_SECRET	飞书应用密钥
FEISHU_DOMAIN	域名, 国内飞书填 feishu, 国际版填 lark
FEISHU_CONNECTION_MODE	连接模式: websocket 或 webhook
FEISHU_ALLOW_ALL_USERS	是否允许所有用户 (true/false)
FEISHU_ALLOWED_USERS	白名单, 逗号分隔的 open_id 列表
FEISHU_GROUP_POLICY	群组策略: open (所有群)、allowlist (仅白名单群)
FEISHU_REQUIRE_MENTION	是否要求 @ 提及才回复 (默认 true)

4.4 DeepSeek API 消费监控

脚本位置 hermes/scripts/deepseek-monitor.py

用途 查询 DeepSeek API 账户余额, 支持阈值告警和 Token 用量统计。

前提条件

- DEEPSEEK_API_KEY 环境变量已设置
- pip install requests

用法

仅查询余额

```
./hermes/scripts/deepseek-monitor.py
```

余额低于 10 CNY 时退出码为 2 (可用于告警链)

```
./hermes/scripts/deepseek-monitor.py --alert 10
```

```
# 从 Hermes session 日志累计 Token 用量
./hermes/scripts/deepseek-monitor.py --track ~/.hermes/sessions
```

API 端点

- GET <https://api.deepseek.com/user/balance>
- 官方文档: <https://api-docs.deepseek.com/zh-cn/api/get-user-balance>

输出示例

DeepSeek API 消费监控

```
状态:          可用
总余额:        55.07 CNY
充值余额:      55.07 CNY
赠金余额:      0.00 CNY
查询时间:      2026-05-15 15:34:08
```

集成到 **Hermes cron** 可用 `no_agent=true` 模式定时运行:

- 每天 9 点推送余额到飞书
- 余额低于 10 元时告警

附录 A 一些碎碎念

i3wm 和 aerospace 是真的好用。窗口最大化直接通过快捷键完成，两个窗口并列操作也可以迅速完成。切换不同的工作区也很迅速，太完美了。简单易用，易上手，减少了许多鼠标的操作，不同费劲的用鼠标拖拽和调整大小，perfect!

真不错!